

Labwork 3: Hello, CUDA!

Duong Tan Binh
Student ID: 2540007

1 Objective

Convert the 512 by 600 RGB image to grayscale on the CPU and GPU, then measure the speedup with different block sizes.

2 Implementation

The image is loaded with `imread` and reshaped to `(pixelCount, 3)`. Both versions calculate the mean of the three channels:

$$\text{gray} = (R + G + B) / 3$$

The CPU uses a for loop. The GPU assigns one pixel to each thread, using `threadIdx.x + blockIdx.x * blockDim.x` as the pixel index. Threads outside the image are skipped. RGB values are converted to integers before addition to avoid overflow, and the result is stored as `uint8`.

3 Timing and results

Times are averaged over five runs using `time.time()`. GPU time includes both data transfers and kernel execution, after a warm-up run. `cuda.synchronize()` waits for the GPU before stopping the timer. File operations and memory allocation are not timed. Speedup is CPU time divided by GPU time.

CPU mean: 89.216 ms

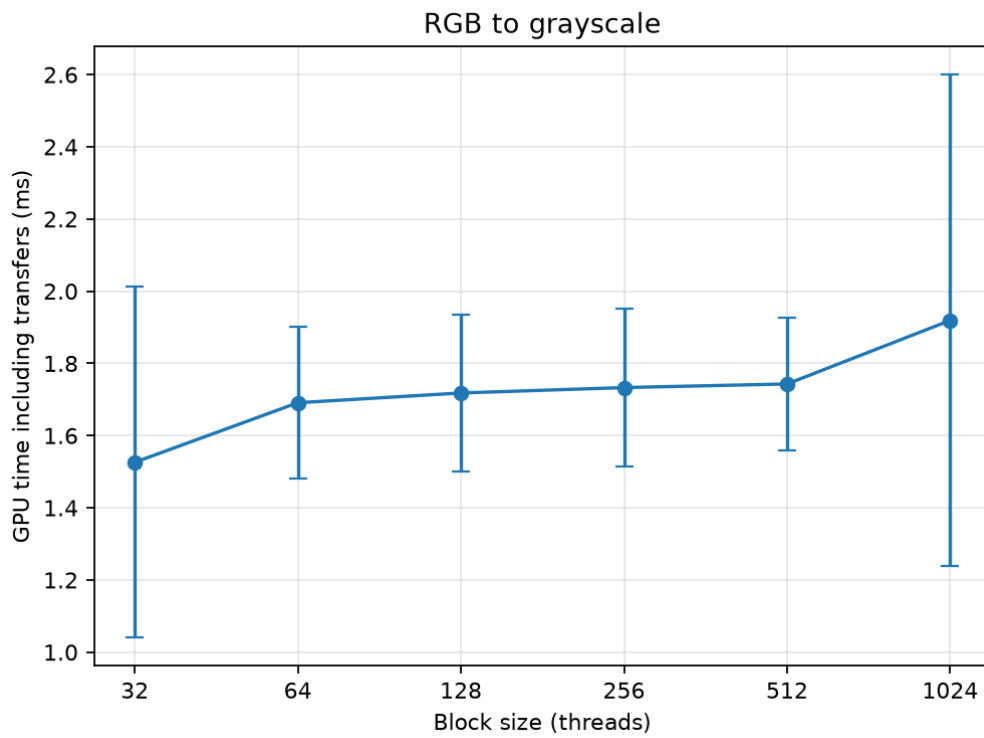
Block size	GPU mean (ms)	Std (ms)	Speedup
32	1.526	0.486	58.46x
64	1.691	0.211	52.77x
128	1.718	0.217	51.94x
256	1.733	0.219	51.49x
512	1.743	0.183	51.19x
1024	1.919	0.681	46.48x

Measurements were taken on an NVIDIA GeForce RTX 5060 Ti. The CPU and GPU images match exactly for all six block sizes.

4 Discussion

The GPU is 46.48 to 58.46 times faster than the CPU loop in this experiment. Block size 32 gives the lowest mean time, 1.526 ms; block size 1024 takes 1.919 ms. Increasing the block size does not reduce the measured time for this image. The error bars overlap, so the difference between block sizes is not conclusive. Data transfer and launch overhead are included in these short timings.

5 Images and timing graph



Error bars show one standard deviation over five runs.



Left to right: input RGB image, CPU grayscale, and GPU grayscale.